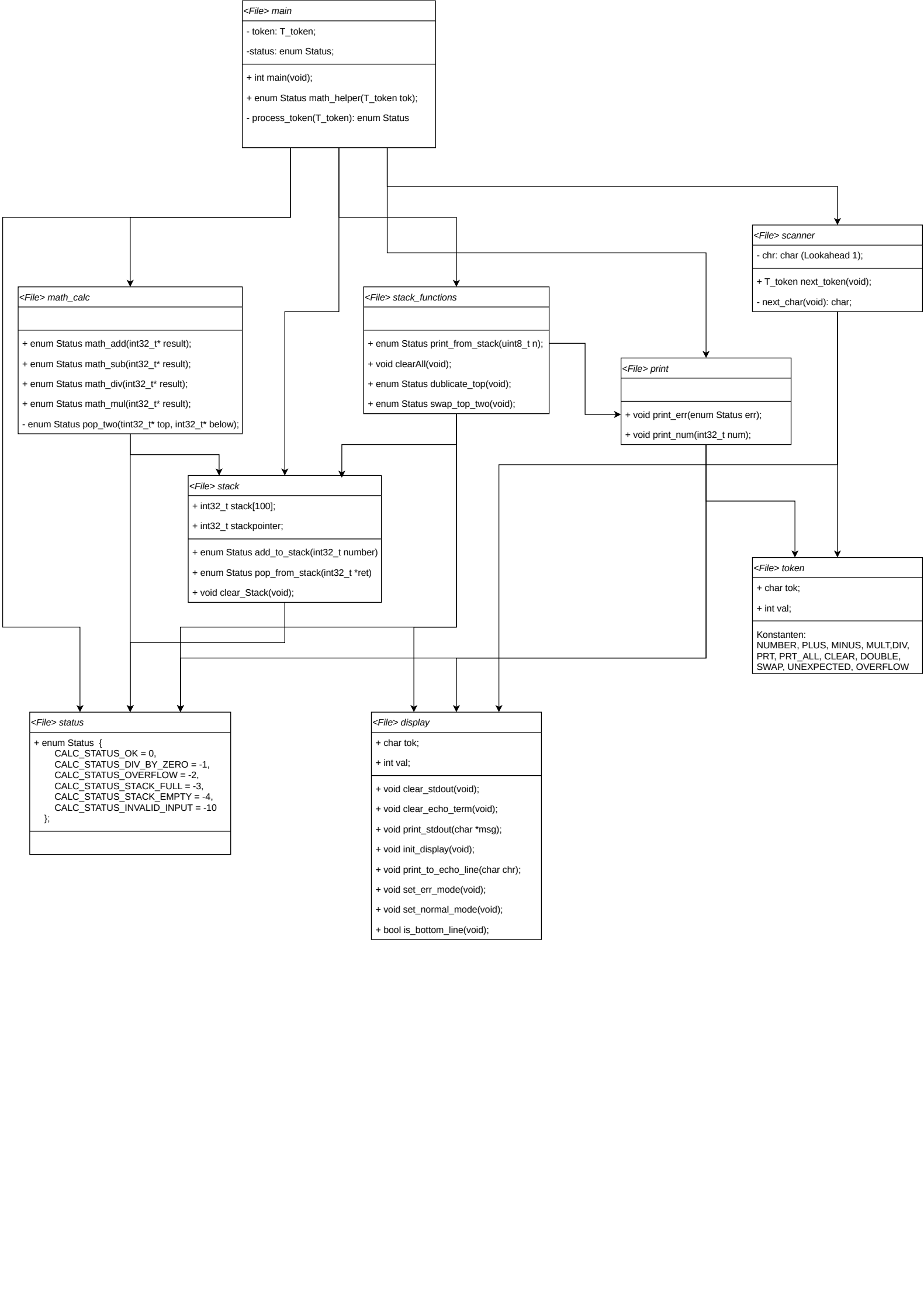
<File> main
- token: T_token;
-status: enum Status;
+ int main(void);
+ enum Status math_helper(T_token tok);
- process_token(T_token): enum Status
<File> scanner
- chr: char (Lookahead 1);
+ T_token next_token(void);
- next_char(void): char;
<File> math_calc
+ enum Status math_add(int32_t* result);
+ enum Status math_sub(int32_t* result);
+ enum Status math_div(int32_t* result);
+ enum Status math_mul(int32_t* result);
- enum Status pop_two(tint32_t* top, int32_t* below);
<File> stack_functions
+ enum Status print_from_stack(uint8_t n);
+ void clearAll(void);
+ enum Status dublicate_top(void);
+ enum Status swap_top_two(void);
<File> print
+ void print_err(enum Status err);
+ void print_num(int32_t num);
<File> stack
+ int32_t stack[100];
+ int32_t stackpointer;
+ enum Status add_to_stack(int32_t number)
+ enum Status pop_from_stack(int32_t *ret)
+ void clear_Stack(void);
<File> token
+ char tok;
+ int val;
Konstanten:
NUMBER, PLUS, MINUS, MULT,DIV,
PRT, PRT_ALL, CLEAR, DOUBLE,
SWAP, UNEXPECTED, OVERFLOW
<File> status
+ enum Status {
CALC_STATUS_OK = 0,
CALC_STATUS_DIV_BY_ZERO = -1,
CALC_STATUS_OVERFLOW = -2,
CALC_STATUS_STACK_FULL = -3,
CALC_STATUS_STACK_EMPTY = -4,
CALC_STATUS_INVALID_INPUT = -10
};
<File> display
+ char tok;
+ int val;
+ void clear_stdout(void);
+ void clear_echo_term(void);
+ void print_stdout(char *msg);
+ void init_display(void);
+ void print_to_echo_line(char chr);
+ void set_err_mode(void);
+ void set_normal_mode(void);
+ bool is_bottom_line(void);